λόγος που η Assembly χαρακτηρίζεται ως γλώσσα χαμηλού επιπέδου (low level). Για παράδειγμα, ο κώδικας που ακολουθεί είναι σε Assembly x86-64 και τυπώνει στην οθόνη τον χαρακτήρα '!'.

```
push    $0x21       # '!'
mov     $1, %rax    # sys_write call number
mov     $1, %rdi    # write to stdout (fd=1)
mov     %rsp, %rsi  # use char on stack
mov     $1, %rdx    # write 1 char
syscall
add     $8, %rsp    # restore sp
```

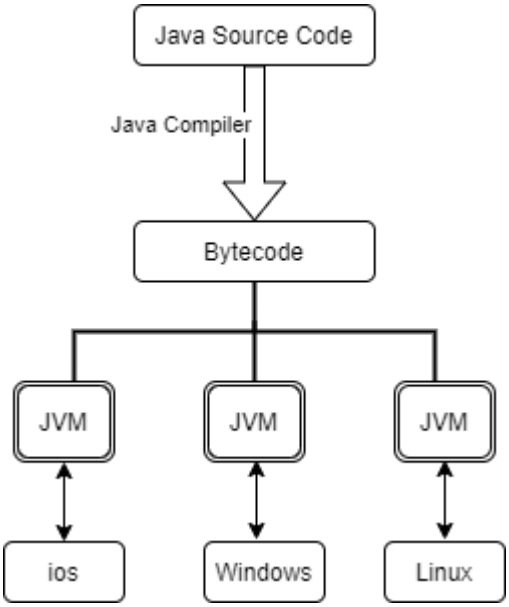
Έτσι προέκυψε η ανάγκη για τις γλώσσες υψηλού επιπέδου (high level programming languages). Ήδη το 1957 κατασκευάστηκε η πρώτη γλώσσα υψηλού επιπέδου, η Fortran της IBM. Οι γλώσσες αυτές είναι ακόμη πιο κοντά στον άνθρωπο σε σχέση με την Assembly και υποστηρίζουν περισσότερες και πιο σύνθετες εντολές. Για παράδειγμα, σε σύγκριση με την Assembly, ο κώδικας Java που εκτυπώνει τον χαρακτήρα '!' είναι System.out.print('!'). Οι γλώσσες υψηλού επιπέδου αυξάνουν δραματικά την παραγωγικότητα κατά την ανάπτυξη εφαρμογών, καθώς ο κώδικας είναι απλούστερος σε σύγκριση με μία γλώσσα μηχανής και Assembly, ενώ δεν απαιτούν να γραφεί διαφορετικό πρόγραμμα για κάθε διαφορετική μηχανή. Ο κώδικας που εκφράζεται σε γλώσσα υψηλού επιπέδου ή Assembly ονομάζεται πηγαίος κώδικας (source code). Σήμερα υπάρχει μεγάλο πλήθος γλωσσών υψηλού επιπέδου με κυριότερους αντιπροσώπους τις C, C++, C#, C objective, Java, Python, Javascript, PHP.

Μεταγλωττιστής: Είναι ευνόητο πως ο πηγαίος κώδικας δεν είναι εκτελέσιμος από τη μηχανή. Ο μεταγλωττιστής (compiler) λαμβάνει ως είσοδο πηγαίο κώδικα και παράγει κώδικα σε γλώσσα μηχανής. Με αυτόν τον τρόπο κωδικοποιείται μια λειτουργία μια φορά σε γλώσσα υψηλού επιπέδου και από αυτήν την κωδικοποίηση μπορεί να παραχθεί εκτελέσιμος κώδικας για κάθε μηχανή που διαθέτει τον αντίστοιχο μεταγλωττιστή. Ένα από τα πλεονεκτήματα του μεταγλωττιστή είναι πως διευκολύνει την προστασία των πνευματικών δικαιωμάτων του κώδικα, καθώς ο κατασκευαστής του λογισμικού δεν διαθέτει στον πελάτη τον πηγαίο αλλά μόνο τον εκτελέσιμο κώδικα. Επιπλέον, διευκολύνει τον προγραμματιστή, καθώς εντοπίζει μια σειρά από συντακτικά κυρίως λάθη κατά τη φάση της μεταγλώττισης (compile time).

Διασυνδέτης: Συχνά, ο πηγαίος κώδικας μίας εφαρμογής αποτελείται από περισσότερα από ένα αρχεία. Κάθε ένα από αυτά τα αρχεία μεταγλωττίζεται χωριστά και για κάθε ένα παράγεται ένα χωριστό αρχείο. Προκειμένου όμως η εφαρμογή να τρέξει, προκύπτει η ανάγκη τα αρχεία που παράγονται από τον μεταγλωττιστή να διασυνδεθούν μεταξύ τους. Την εργασία αυτή αναλαμβάνει ο διασυνδέτης (linker).

Διερμηνευτής: Ο διερμηνευτής (Interpreter) επίσης μεταφράζει τον πηγαίο κώδικα σε εκτελέσιμο αλλά το κάνει κατά τον χρόνο εκτέλεσης (run time), δηλαδή ο διερμηνευτής κατά τον χρόνο εκτέλεσης απαιτεί διαθέσιμο τον πηγαίο κώδικα που μεταφράζει και εκτελεί εντολή προς εντολή. Επομένως ο διερμηνευτής δεν εντοπίζει τα συντακτικά λάθη παρά μόνο κατά τον χρόνο εκτέλεσης του προγράμματος, ενώ δεν παρέχει προστασία των πνευματικών δικαιωμάτων του πηγαίου κώδικα. Επιπλέον, ένα πρόγραμμα σε γλώσσα διερμηνευτή απαιτεί μεγαλύτερο χρόνο εκτέλεσης από το ίδιο πρόγραμμα μεταγλωττισμένο.

Εικονική μηχανή: Όπως αναφέραμε προηγουμένως, ο μεταγλωττιστής μεταφράζει τον πηγαίο κώδικα σε εκτελέσιμο. Πρόκειται για περίπλοκο λογισμικό και η υλοποίησή του απαιτεί αρκετή εργασία και υψηλό κόστος. Όταν μάλιστα θέλουμε τα προγράμματα που αναπτύσσουμε σε μια γλώσσα να είναι εκτελέσιμα σε πολλές πλατφόρμες, είμαστε αναγκασμένοι να υλοποιήσουμε έναν διαφορετικό μεταγλωττιστή για κάθε πλατφόρμα. Μια εναλλακτική λύση είναι η αξιοποίηση εικονικής μηχανής. Στην περίπτωση αυτή έχουμε συνδυασμό ενός μεταγλωττιστή με μια εικονική μηχανή. Ο μεταγλωττιστής όμως δεν παράγει κώδικα εκτελέσιμο απευθείας από τη μηχανή. Αντίθετα, παράγει έναν ενδιάμεσο κώδικα, γνωστό ως bytecode, ο οποίος αποτελεί είσοδο για την εικονική μηχανή που με τη σειρά της παράγει τον εκτελέσιμο κώδικα. Επιπλέον, ο ενδιάμεσος κώδικας δεν διασυνδέεται από έναν τυπικό διασυνδέτη αλλά από τον επονομαζόμενο φορτωτή κλάσης (class loader). Η τεχνολογία αυτή έχει υιοθετηθεί από την Java με την υλοποίηση της Java Virtual Machine ή JVM για συντομία.



Σχήμα 1.1: *Η λειτουργία της εικονικής μηχανής της Java*

Όπως φαίνεται στο Σχήμα 1.1, ο πηγαίος κώδικας Java μεταφράζεται σε Bytecode. Ο κώδικας Bytecode είναι ανεξάρτητος από τη φυσική μηχανή στην οποία θέλουμε να τρέξει. Στη συνέχεια, ο ενδιάμεσος κώδικας τρέχει σε διάφορα λειτουργικά μέσω της εικονικής μηχανής. Επομένως δεν χρειαζόμαστε έναν μεταγλωττιστή για κάθε λειτουργικό σύστημα. Αντίθετα, ένας μεταγλωττιστής για όλα τα λειτουργικά μαζί με μια εικονική μηχανή για κάθε ένα λειτουργικό αρκούν. Η εικονική μηχανή όμως είναι πολύ απλούστερη σε σχέση με τον μεταγλωττιστή και έτσι η ανάπτυξή της για κάθε λειτουργικό είναι πολύ οικονομικότερη από την ανάπτυξη μεταγλωττιστή.

Ο μεταγλωττιστής Just In Time (JIT): Ο μεταγλωττιστής JIT εκτελείται από την εικονική μηχανή και η δουλειά του είναι η μεταγλώττιση του bytecode σε εκτελέσιμο. Ο μεταγλωττιστής JIT δεν ενδιαφέρεται για συντακτικά λάθη καθώς αυτά έχουν εντοπιστεί και διορθωθεί κατά τη μεταγλώττιση του πηγαίου κώδικα. Επιπλέον, δεν μεταφράζει άμεσα όλον τον bytecode σε εκτελέσιμο. Αντίθετα, μεταφράζει μόνο τα τμήματα που είναι αναγκαία ανάλογα με τις κλήσεις που περιλαμβάνει ο κώδικας. Έτσι η εκτέλεση και η μετάφραση του κώδικα γίνονται παράλληλα. Το αποτέλεσμα είναι πως η ταχύτητα εκτέλεσης του bytecode προσεγγίζει την ταχύτητα εκτέλεσης του κώδικα μηχανής. Ας σημειωθεί πως στη βιβλιογραφία ο JIT συχνά αναφέρεται ως μεταγλωττιστής, ενώ κάποιες φορές αναφέρεται ως διερμηνευτής (Java Interpreter). Στην πράξη πρόκειται για κάτι ενδιάμεσο. Ο JIT μετατρέπει τον bytecode σε εκτελέσιμο κατά τον χρόνο εκτέλεσης που είναι χαρακτηριστικό του διερμηνευτή. Από την άλλη πλευρά όμως, ένα τμήμα κώδικα που έχει ήδη μεταφραστεί δεν απαιτείται να μεταφραστεί και πάλι όταν χρησιμοποιείται κατά την τρέχουσα εκτέλεση του προγράμματος για δεύτερη φορά.

JRE και JDK: Το Java Runtime Environment (JRE) περιλαμβάνει την JVM και επιπλέον κάποιες βιβλιοθήκες της Java που είναι απαραίτητες για να εκτελεστεί μια εφαρμογή Java. Το Java Development Kit (JDK) περιλαμβάνει την JRE και μια σειρά από εργαλεία απαραίτητα για την ανάπτυξη πηγαίου κώδικα σε Java, π.χ. ο μεταγλωττιστής που μεταφράζει από πηγαίο κώδικα σε bytecode.

Απολαθοποιητής (Debugger): Ο απολαθοποιητής είναι ένα λογισμικό που μας βοηθάει να εντοπίζουμε λάθη στον κώδικα. Ο απολαθοποιητής μας δίνει τη δυνατότητα να εκτελούμε το πρόγραμμά μας ή κάποιο τμήμα του, βήμα-βήμα, και σε κάθε βήμα να ελέγχουμε το περιεχόμενο των μεταβλητών. Έτσι διευκολυνόμαστε να εντοπίσουμε σε ποιο σημείο ακριβώς συμβαίνει το λάθος και να το διορθώσουμε.

Ολοκληρωμένα Περιβάλλοντα Ανάπτυξης: Τα Ολοκληρωμένα Περιβάλλοντα Ανάπτυξης (Integrated Development Environment ή IDE για συντομία) είναι λογισμικά που σκοπό έχουν να μας βοηθήσουν στην ανάπτυξη εφαρμογών. Σήμερα είναι διαθέσιμα πολλά τέτοια περιβάλλοντα για ποικίλες γλώσσες.

Παραδείγματα αποτελούν το NetBeans, το Eclipse, το Visual Studio, το IntelliJ, το PyCharm κ.ά. Κατά τη συγγραφή αυτού του βιβλίου χρησιμοποιήθηκε το NetBeans, έκδοση 12.5.

Διεπαφή Προγραμματισμού Εφαρμογών: Η Διεπαφή Προγραμματισμού Εφαρμογών (Application Programming Interface ή API για συντομία) είναι μια βιβλιοθήκη κώδικα που μπορεί να χρησιμοποιηθεί για την ανάπτυξη εφαρμογών.

1.3 Βασικές αρχές σχεδίασης της Java

Η Java σχεδιάστηκε στη βάση των ακόλουθων αρχών [8]:

Απλότητα, Αντικειμενοστρέφεια και Οικειότητα
Η Java σχεδιάστηκε να είναι απλή στη χρήση της ώστε οι προγραμματιστές να γίνουν σύντομα παραγωγικοί. Πιθανότατα σε αυτήν την αρχή βασίστηκε το απλό μοντέλο που υιοθετεί η Java στη διαχείριση της μνήμης. Επιπλέον είναι χρήσιμο, οι προγραμματιστές να είναι ήδη εξοικειωμένοι με τις δομές της Java. Έτσι αποφασίστηκε να ομοιάζουν οι δομές της Java με αυτές της C++. Επίσης, η Java να υποστηρίζει ολοκληρωμένα το αντικειμενοστρεφές μοντέλο.

Αξιοπιστία και Ασφάλεια
Η Java σχεδιάστηκε για να υλοποιεί εφαρμογές υψηλής αξιοπιστίας. Τα χαρακτηριστικά της γλώσσας καθοδηγούν τους προγραμματιστές να υιοθετούν πρακτικές που καθιστούν τους κώδικες αξιόπιστους και σταθερούς. Η ασφάλεια αποτέλεσε κρίσιμο ζητούμενο κατά τον σχεδιασμό. Ενσωματωμένα στη γλώσσα χαρακτηριστικά καθιστούν τις εφαρμογές της Java δύσκολο στόχο στις επιθέσεις από κακόβουλο λογισμικό.

Φορητότητα
Οι εφαρμογές σε Java θα πρέπει να εκτελούνται χωρίς πρόβλημα σε οποιαδήποτε πλατφόρμα. Η αρχή αυτή επιτυγχάνεται με την υιοθέτηση της αρχιτεκτονικής της εικονικής μηχανής. Σε οποιαδήποτε πλατφόρμα υπάρχει η JVM, οι εφαρμογές Java εκτελούνται χωρίς πρόβλημα.

Ταχύτητα
Η Java επιτυγχάνει πολύ υψηλές ταχύτητες. Σε αυτό συμβάλλουν πολλά χαρακτηριστικά της γλώσσας όπως η αυτόματη διαχείριση της μνήμης (ενότητα 3.8.1), η έξυπνη μετάφραση του κώδικα από τον JIT αλλά και ο καθορισμός των τύπων των μεταβλητών κατά τον χρόνο μεταγλώττισης (ενότητα 3.2).

Πολυνηματικός και δυναμικός χαρακτήρας και διερμήνευση
Ο διερμηνευτής JIT της Java μπορεί να εκτελέσει κώδικα σε οποιαδήποτε μηχανή είναι εγκατεστημένη η εικονική μηχανή. Η Java σχεδιάστηκε εξαρχής ώστε να υποστηρίζει πολυνηματισμό, δηλαδή τη δυνατότητα, τμήματα του κώδικα να εκτελούνται παράλληλα από τον ίδιο επεξεργαστή. Τέλος, ο δυναμικός χαρακτήρας της Java αναφέρεται στη δυναμική σύνδεση που γίνεται κατά τον χρόνο εκτέλεσης. Σε αυτήν τη φάση ενδέχεται να συνδεθούν, ανάλογα με τις ανάγκες, κώδικες ακόμη και από απομακρυσμένες τοποθεσίες του διαδικτύου.

1.4 Τεχνολογίες της Java

Οι τυπικές γλώσσες προγραμματισμού παρέχουν στον προγραμματιστή μια σειρά από βασικές λειτουργίες όπως είναι η δυνατότητα ανάπτυξης επαναληπτικών δομών, οι δομές επιλογής, η δημιουργία αντικειμένων, η διαχείριση της μνήμης, κλπ. Τις δυνατότητες αυτές προσφέρει και η C++. Όμως κατά τη δημιουργία των σύγχρονων εφαρμογών απαιτούνται και άλλες πέραν των βασικών αυτών δυνατοτήτων, π.χ. ο προγραμματισμός της διεπαφής χρήστη (user interface). Έτσι αν αναπτύσσουμε μια εφαρμογή με C++ και χρειαζόμαστε παραθυρική διεπαφή χρήστη, θα πρέπει να αξιοποιήσουμε κατάλληλες βιβλιοθήκες τρίτων που υποστηρίζουν την ανάπτυξη τέτοιων διεπαφών, καθώς η ίδια η C++ δεν παρέχει αυτήν τη δυνατότητα. Αντίθετα η Java υποστηρίζει στο βασικό πλαίσιο και τον προγραμματισμό της διεπαφής χρήστη και πολλές άλλες δυνατότητες όπως θα δούμε αμέσως παρακάτω. Το σύνολο των βασικών δυνατοτήτων στην Java θεωρείται ότι συνιστά τον πυρήνα της γλώσσας (Java core) πάνω στον οποίο αναπτύσσονται μια σειρά από εξειδικευμένες τεχνολογίες κατάλληλες για την ανάπτυξη ποικίλων εφαρμογών.

Βάσεις Δεδομένων: Η σύνδεση και επικοινωνία με βάση δεδομένων γίνεται με βιβλιοθήκες ενσωματωμένες στο βασικό πλαίσιο της Java. Ακόμη περαιτέρω, η Java διαθέτει τη δική της Βάση Δεδομένων (Java DB).

Ενσωματωμένα Συστήματα: Με το πλαίσιο Java SE Embedded [9], η Java υποστηρίζει την ανάπτυξη αξιόπιστων και φορητών εφαρμογών υψηλής λειτουργικότητας για τα περισσότερα από τα σύγχρονα ενσωματωμένα συστήματα.

Υπηρεσίες αναγνώρισης και εξουσιοδότησης: Το πλαίσιο αναγνώρισης και εξουσιοδότησης (Java Authentication and Authorization Services) χρησιμοποιείται για να προσδιορίζεται ο χρήστης της εφαρμογής ώστε να εξουσιοδοτούνται να εκτελούν συγκεκριμένες εργασίες μόνο όσοι διαθέτουν κατάλληλα δικαιώματα [10, 11].

Διεθνοποίηση: Με το πλαίσιο διεθνοποίησης (Internationalization) [12] μπορούμε εύκολα να αλλάξουμε τη γλώσσα και άλλα τοπικά χαρακτηριστικά μιας εφαρμογής, π.χ. το ημερολόγιο.

Προσαρμοσμένη δικτύωση: Με την υποστήριξη της προσαρμοσμένης δικτύωσης (Custom Networking) [13], η Java μας δίνει τη δυνατότητα ανάπτυξης εφαρμογών που χρησιμοποιούν και αλληλεπιδρούν με πόρους στο Διαδίκτυο και στον Παγκόσμιο Ιστό.

Διανομή εφαρμογών: Με τη διανομή εφαρμογών (Deployment), η Java μας δίνει τη δυνατότητα να παράγουμε και να διανέμουμε μέσω διαδικτύου εμπλουτισμένες εφαρμογές (Rich Internet Applications ή RIA), δηλαδή εφαρμογές που διαθέτουν τα ισχυρά χαρακτηριστικά των εφαρμογών επιτραπέζιου υπολογιστή αλλά διανέμονται μέσω διαδικτύου [14].

Κανονικές εκφράσεις: Η Java υποστηρίζει πλήρως τις κανονικές εκφράσεις (Regular Expressions). Πρόκειται για συμβολοσειρές που καθορίζουν ένα μοτίβο αναζήτησης [15, 16].

Διεπαφή προγραμματισμού εφαρμογών επεξεργασίας XML εγγράφων: Η διεπαφή αυτή (API for XML Processing) παρέχει στις εφαρμογές της Java τη δυνατότητα εύκολης επεξεργασίας XML εγγράφων.

Διεπαφή προγραμματισμού εφαρμογών email και υπηρεσία διαχείρισης μηνυμάτων: Οι διεπαφές αυτές (JavaMail API και Java Message Service API) παρέχουν τη δυνατότητα ανάπτυξης εφαρμογών διαχείρισης και ανταλλαγής μηνυμάτων.

Πλαίσιο ενεργοποίησης JavaBeans: Το πλαίσιο αυτό (JavaBeans Activation Framework, JAF) παρέχει τη δυνατότητα αναγνώρισης του τύπου των δεδομένων. Για παράδειγμα, ένα πρόγραμμα περιήγησης στο διαδίκτυο, λαμβάνει μια σειρά από δεδομένα που συνιστούν μια εικόνα JPEG. Με το πλαίσιο ενεργοποίησης μπορούμε να αναγνωρίσουμε ότι πρόκειται για εικόνα JPEG και να τη διαχειριστούμε κατάλληλα.

Διεπαφή Servlet: Παρέχει λειτουργίες που συνεργάζονται με έναν εξυπηρετητή διαδικτύου (Web Server) και τον βοηθούν στη διαχείριση των αιτημάτων των πελατών (client) [17].

Σελίδες Εξυπηρετητή: Η τεχνολογία αυτή (JavaServer Pages Technology) παρέχει τη δυνατότητα δυναμικής διαμόρφωσης ιστοσελίδων για τις εφαρμογές του Παγκόσμιου Ιστού (Web Applications).

Βιβλιοθήκη ετικετών: Η βιβλιοθήκη ετικετών (JavaServer Pages Standard Tag Library) παρέχει με τη μορφή απλών ετικετών (tags) βασική λειτουργικότητα για εφαρμογές του Παγκόσμιου Ιστού, π.χ. ετικέτες για επαναληπτικές δομές, για δομές επιλογής, για διαχείριση XML εγγράφων [18].

Τεχνολογία JavaServer Faces Technology: Η τεχνολογία αυτή απλοποιεί την ενσωμάτωση διεπαφών χρήστη σε εφαρμογές του Παγκόσμιου Ιστού [19].

JavaFx: Το πλαίσιο αυτό είναι κατάλληλο για την ανάπτυξη εμπλουτισμένων εφαρμογών διαδικτύου. Ενσωματώνει τη δυνατότητα ανάπτυξης επιτραπέζιων εφαρμογών με παραθυρική διεπαφή [20]. Ας σημειωθεί

ότι είναι το νεότερο από τα τρία πλαίσια που διαθέτει η Java για ανάπτυξη παραθυρικών εφαρμογών. Τα άλλα δύο είναι το Abstract ToolKit και το Swing.

Άλλα χαρακτηριστικά της Java: Η Java είναι η βασική γλώσσα ανάπτυξης εφαρμογών Android. Επίσης, η JavaFx σε συνεργασία με την τεχνολογία Gluon μπορεί να κτίσει παραθυρικές εφαρμογές και να τις μεταφέρει στις πλατφόρμες κινητών Android και iOS. Ένα άλλο σημαντικό χαρακτηριστικό της Java είναι η συμβατότητα μεταξύ των εκδόσεων (Backwards compatibility). Πρόκειται για αναγκαίο χαρακτηριστικό προκειμένου οι επιχειρήσεις να επενδύουν για ανάπτυξη σε μια γλώσσα. Επιπλέον, η Java είναι ανοιχτό λογισμικό (Open Source). Αυτό διευκολύνει την ανάπτυξη βιβλιοθηκών τρίτων σε μορφή ανοιχτού κώδικα. Για παράδειγμα, αν κάποιος θελήσει να αναπτύξει μια εφαρμογή οπτικής αναγνώρισης χαρακτήρων (Optical Character Recognition, OCR), μπορεί εύκολα κατάλληλες βιβλιοθήκες σε Java. Αντίστοιχες βιβλιοθήκες σε άλλες γλώσσες κοστίζουν συνήθως μερικές χιλιάδες Ευρώ.

Βιβλιογραφία

[1] Μ. Δερτούζος, Η ΑΝΟΛΟΚΛΗΡΩΤΗ ΕΠΑΝΑΣΤΑΣΗ. Λιβάνης, 2001. Accessed: Sep. 14, 2021. [Online]. Available: https://www.politeianet.gr/books/9789601403755-dertouzos-michalis-libanis-i-anolokliroti-epanastasi-23070

[2] Y. Ilyin, "Digital realities in 2045: future technologies and security issues," Kaspersky daily, 2015. https://www.kaspersky.com/blog/digital-realities-in-2045-future-technologies-and-security-issues/15047/ (accessed Sep. 14, 2021).

[3] S. Walter and M. Kenrick, Java μια εισαγωγή στην επίλυση προβλημάτων και στον Προγραμματισμό, 7η έκδοση. Εκδόσεις Τζιόλα, 2016.

[4] "Java (software platform)," Wikipedia. Apr. 24, 2021. Accessed: Jun. 13, 2021. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Java_(software_platform)&oldid=1019658740

[5] "Java (programming language)," Wikipedia. Jun. 10, 2021. Accessed: Jun. 13, 2021. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Java_(programming_language)&oldid=1027807155

[6] S. J. Vaughan-Nichols, "A Google Android and Java history lesson," ZDNet. https://www.zdnet.com/article/a-google-android-and-java-history-lesson/ (accessed Jun. 13, 2021).

[7] Taniani, "What Is JavaFX?," Social net: Databases Oracle, MySQL, programming SQL, Java, APEX, administration, Jul. 03, 2018. https://oracle-patches.com/en/coding/what-is-javafx (accessed Jun. 13, 2021).

[8] "The Java Language Environment." https://www.oracle.com/java/technologies/introduction-to-java.html (accessed Jun. 13, 2021).

[9] "Oracle Java SE Embedded Overview." https://www.oracle.com/java/technologies/javase-embedded/javase-embedded.html (accessed Jun. 13, 2021).

[10] "Java Platform, Standard Edition Security Developer's Guide." https://docs.oracle.com/javase/9/security/java-authentication-and-authorization-service-jaas1.htm#JSSEC-GUID-76141D9C-03F1-40D8-93D8-DFFC2143A2CA (accessed Jun. 13, 2021).

[11] "JAAS Reference Guide." https://docs.oracle.com/javase/7/docs/technotes/guides/security/jaas/JAASRefGuide.html#Introduction (accessed Jun. 13, 2021).

[12] "Java SE 8 Internationalization (I18n) Developer Guides." https://docs.oracle.com/javase/8/docs/technotes/guides/intl/index.html (accessed Jun. 13, 2021).

[13] "Trail: Custom Networking (The Java™ Tutorials)." https://docs.oracle.com/javase/tutorial/networking/index.html (accessed Jun. 13, 2021).

[14] "Trail: Deployment (The Java™ Tutorials)." https://docs.oracle.com/javase/tutorial/deployment/index.html (accessed Jun. 13, 2021).

[15] "Java Regular Expressions." https://www.w3schools.com/java/java_regex.asp (accessed Jun. 13, 2021).

[16] A. Srivastava, Java 9 Regular Expressions: A hands-on guide to implement zero-length assertions, back-references, quantifiers, and many more. Packt Publishing, 2017.

[17] "Introduction to Java Servlets," GeeksforGeeks, May 28, 2018. https://www.geeksforgeeks.org/introduction-java-servlets/ (accessed Jun. 14, 2021).

[18] "JavaServer Pages Standard Tag Library." https://www.oracle.com/java/technologies/java-server-tag-library.html (accessed Jun. 14, 2021).

[19] “JavaServer Faces (JSF) Tutorial - Tutorialspoint.” https://www.tutorialspoint.com/jsf/index.htm (accessed Jun. 14, 2021).

[20] The Definitive Guide to Modern Java Clients with JavaFX - Cross-Platform Mobile and Cloud Development | Stephen Chin | Apress. Apress, 2019. Accessed: Jun. 14, 2021. [Online]. Available: https://www.apress.com/gp/book/9781484249253

Κεφάλαιο 2

Σύνοψη
Σε αυτήν την ενότητα μαθαίνουμε καταρχάς πώς να εγκαθιστούμε την Java και το περιβάλλον ανάπτυξης NetBeans. Στη συνέχεια παρουσιάζουμε και αναλύουμε ένα απλό εισαγωγικό πρόγραμμα σε Java. Στο πλαίσιο αυτό γίνεται εισαγωγική παρουσίαση των εννοιών: πακέτο (package), η main ως κύρια είσοδος στο πρόγραμμα, το υποπρόγραμμα ως void συνάρτηση χωρίς παραμέτρους και η βασική έξοδος στην οθόνη. Επιπλέον, παρουσιάζονται οι βασικές συμβάσεις ονοματολογίας.

Προαπαιτούμενη γνώση
Δεν απαιτούνται ειδικές γνώσεις για αυτήν την ενότητα.

Λέξεις κλειδιά
Περιβάλλον ανάπτυξης, Κλάση, Συνάρτηση, Ονοματολογία, Βασική Έξοδος.

2. Εγκατάσταση και το πρώτο πρόγραμμα

Όπως στις περισσότερες γλώσσες προγραμματισμού, έτσι και στην Java, ο πηγαίος κώδικας έχει τη μορφή απλού κειμένου. Επομένως μπορεί κανείς να γράψει πρόγραμμα χρησιμοποιώντας οποιονδήποτε απλό επεξεργαστή κειμένου (text editor). Στη συνέχεια, εφόσον έχει εγκαταστήσει την Java, μπορεί να καλέσει τον μεταγλωττιστή της, να μεταγλωττίσει τον πηγαίο κώδικα και στη συνέχεια να καλέσει την εικονική μηχανή για να το τρέξει. Αυτή εν ολίγοις είναι η διαδικασία που ακολουθείτο μέχρι τις αρχές της δεκαετίας του 2000. Ο κώδικας όμως είναι χρήσιμο να παρουσιάζεται κατά τρόπο που να είναι ευανάγνωστος. Σε αυτό συμβάλλει η κατάλληλη στοίχιση των γραμμών των εντολών αλλά και ο κατάλληλος χρωματισμός των διαφορετικών στοιχείων του κώδικα. Η στοίχιση που απαιτείται είναι αρκετά περίπλοκη, καθώς οφείλει να ακολουθεί τις πολύπλοκες δομές του κώδικα. Αντίστοιχα, περίπλοκος είναι και ο χρωματισμός του κώδικα. Οι απλοί επεξεργαστές κειμένου δεν παρέχουν ούτε τη δυνατότητα αυτόματης στοίχισης ούτε τη δυνατότητα αυτόματου χρωματισμού.

Έτσι, προέκυψε η ανάγκη να αναπτυχθούν περιβάλλοντα εξειδικευμένα για συγγραφή και διαχείριση πηγαίου κώδικα. Σύντομα τα περιβάλλοντα, γνωστά ως Ολοκληρωμένα Περιβάλλοντα Ανάπτυξης (Integrated Development Environments ή IDE), ενσωμάτωσαν πλήθος δυνατοτήτων που σκοπό έχουν να διευκολύνουν την ανάπτυξη του λογισμικού.

Τα περισσότερα σύγχρονα IDE υποστηρίζουν πλήθος από γλώσσες προγραμματισμού. Ωστόσο, συνήθως είναι πιο συγγενικά με κάποιες από αυτές. Η εγκατάστασή τους μπορεί να περιλαμβάνει και την αυτόματη εγκατάσταση των συγγενικών γλωσσών προγραμματισμού. Παρέχουν τη δυνατότητα οργάνωσης τόσο των αρχείων του κώδικα σε έργα (projects) αλλά και της εσωτερικής οργάνωσης κάθε αρχείου κώδικα ξεχωριστά. Επιπλέον, αυτοματοποιούν τις διαδικασίες που απαιτούνται ώστε ο πηγαίος κώδικας να μετατραπεί σε εκτελέσιμο και να τρέξει, ενώ παρέχουν συσκευές εξόδου όπου ο προγραμματιστής μπορεί να δει άμεσα το αποτέλεσμα της εκτέλεσης του κώδικα που έγραψε. Επίσης, τα σύγχρονα IDE διαθέτουν ενσωματωμένους απολαθοποιητές και διευκολύνουν τη διαδικασία απολαθοποίησης.

Σήμερα, διατίθεται πλήθος από IDE. Στα πλαίσια αυτού του συγγράμματος όμως χρησιμοποιούμε το NetBeans, έκδοση 12.5, που είναι το συγγενικό IDE της Java.

Στη συνέχεια αυτού του κεφαλαίου, αφού εγκαταστήσουμε το NetBeans μαζί με την Java, παρουσιάζουμε και αναλύουμε το πρώτο μας πρόγραμμα. Μετά εισάγουμε τον αναγνώστη στην έννοια του απλού υποπρογράμματος ή ρουτίνας, επισημαίνουμε βασικές συμβάσεις ονοματολογίας που πρέπει απαραίτητα να ακολουθούμε κατά τη συγγραφή των προγραμμάτων και τέλος παρέχουμε αναγκαίες βασικές πληροφορίες σε σχέση με την εμφάνιση μηνυμάτων από το πρόγραμμα προς τον χρήστη.

2.1 Εγκατάσταση NetBeans και JDK

Μεταβείτε στην τοποθεσία https://netbeans.apache.org/download/nb125/nb125.html και κατεβάστε τον κατάλληλο installer ανάλογα με το λειτουργικό σύστημά σας. Τρέξτε τον installer. Δεχτείτε τις προτεινόμενες επιλογές κατά τη διάρκεια της εγκατάστασης. Με την ολοκλήρωση της εγκατάστασης θα έχετε διαθέσιμα, το

περιβάλλον ανάπτυξης, την JSE, την JEE, τις HTML και Javascript και την PHP. Θα έχετε έτσι ένα ολοκληρωμένο περιβάλλον ανάπτυξης εφαρμογών. Βρείτε το εικονίδιο του NetBeans στην επιφάνεια εργασίας και τρέξτε το. Σε περίπτωση που έχετε ήδη παλαιότερη έκδοση του NetBeans εγκατεστημένη, θα ερωτηθείτε αν επιθυμείτε να κάνετε εισαγωγή των ρυθμίσεών της στη νέα έκδοση. Απαντήστε κατάλληλα. Εφόσον οι ρυθμίσεις στην προηγούμενη έκδοση είναι ορθές, η εισαγωγή τους θα σας φανεί πολλή χρήσιμη. Ας σημειωθεί πως αυτός ο installer εγκαθιστά την έκδοση NetBeans 1.25 με το JDK 17 που αποτελούν τις τελευταίες εκδόσεις και για τα δύο προϊόντα κατά τον χρόνο συγγραφής αυτού του κειμένου.

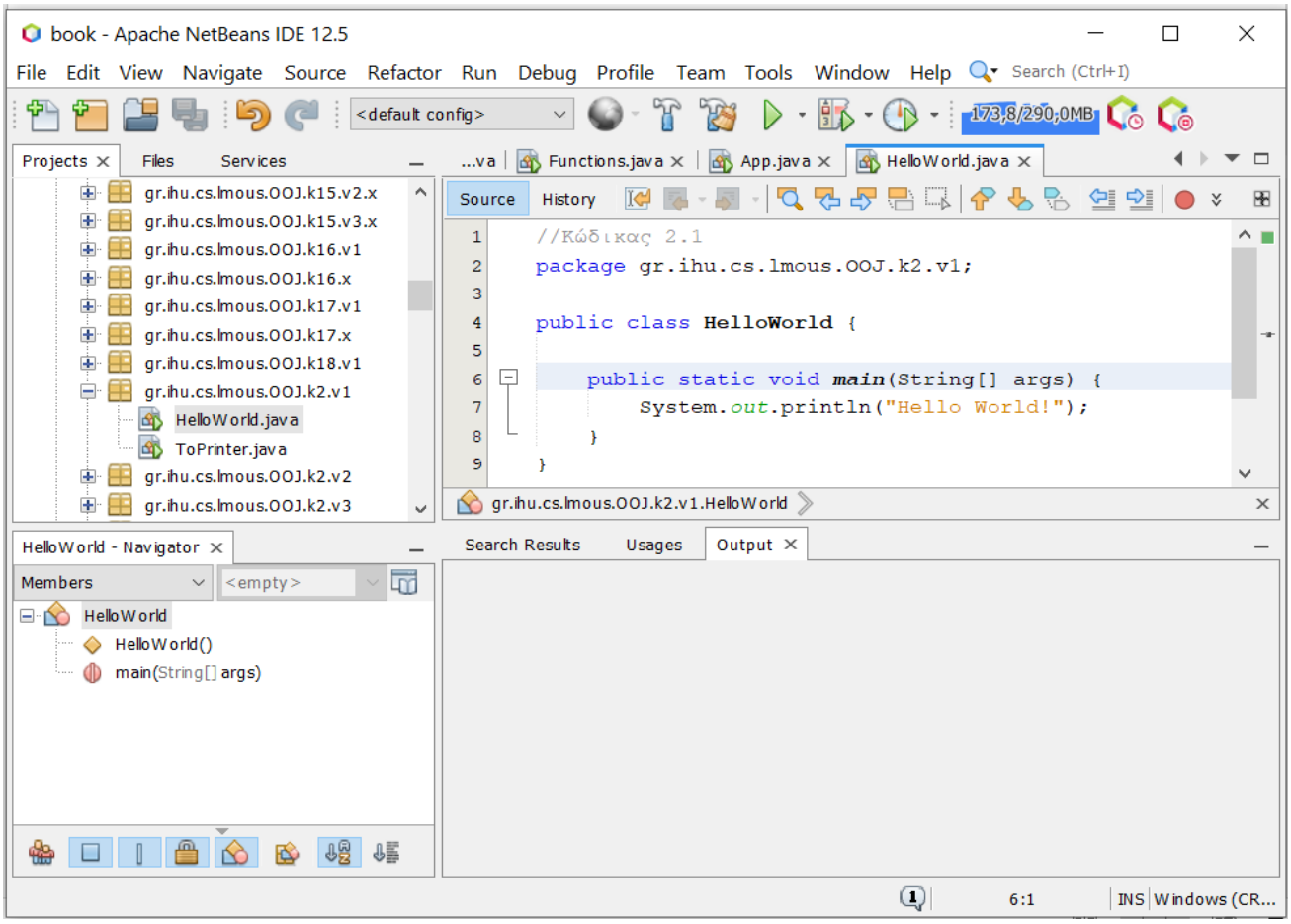
Είστε έτοιμοι να ξεκινήσετε την περιπέτειά σας στον Προγραμματισμό.

2.2 Το πρώτο Πρόγραμμα

Μεταβείτε στο μενού File του NetBeans και επιλέξτε New Project. Στο παράθυρο που θα ανοίξει, επιλέξτε Categories/Java with Ant και Projects/Java Class Library. Πατήστε Next. Στο παράθυρο που θα εμφανιστεί δώστε κατάλληλο όνομα στο project, καθορίστε την τοποθεσία στην οποία θα εμφανιστεί, πατήστε Finish και το project σας έχει δημιουργηθεί. Μπορείτε να καθορίσετε το όνομα της επιλογής σας για το project.

Διαμορφώστε το περιβάλλον εργασίας όπως φαίνεται στην εικόνα 1. Σε αυτήν τη διαμόρφωση περιλαμβάνονται τρία τμήματα. Στα αριστερά είναι τοποθετημένο πλαίσιο με τρία παράθυρα, Projects, Files και Services. Δεξιά επάνω, ο editor, δηλαδή το παράθυρο στο οποίο γράφουμε τον κώδικά μας και δεξιά κάτω το Output, δηλαδή το παράθυρο στο οποίο εμφανίζεται η έξοδος του προγράμματός μας. Αν κάποιο από αυτά τα τμήματα δεν φαίνεται, μεταβείτε στο μενού Window του NetBeans και ανοίξτε το.

Μεταβείτε στο τμήμα Projects, επεκτείνεται το project σας πατώντας στο + που βρίσκεται στα αριστερά του και θα δείτε τους φακέλους του project. Όπως βλέπετε στο project περιλαμβάνονται δύο φάκελοι, ο Source Packages και ο Libraries. Ανοίξτε τον φάκελο Source Packages. Θα διαπιστώσετε πως περιλαμβάνει έναν φάκελο που ονομάζεται Default Package. Ο φάκελος αυτός συνιστά έναν χώρο ή ένα πακέτο όπως λέγεται στο οποίο μπορούμε να τοποθετήσουμε πηγαίους κώδικες. Γενικά τα αρχεία με τους κώδικες τα τοποθετούμε σε πακέτα. Ειδικά όμως η χρήση του Default Package συνίσταται να αποφεύγεται. Αντίθετα, θα πρέπει να δημιουργούμε πακέτα ανάλογα με τις ανάγκες κάθε project. Κάντε δεξί κλικ στο όνομα του project. Από το μενού που θα ανοίξει, επιλέξτε New\Java Package. Δώστε ένα όνομα στο πακέτο και πατήστε finish. Θα διαπιστώσετε πως ο φάκελος Default Package έχει εξαφανισθεί ενώ έχει εμφανισθεί ένας φάκελος με το όνομα που δώσατε στο πακέτο σας. Μέσα σε αυτόν θα τοποθετηθεί το πρώτο πρόγραμμά μας.



***Εικόνα 2.1**: Διαμόρφωση πλαισίου εργασίας:*

Κάντε δεξί κλικ στο πακέτο που δημιουργήσατε, επιλέξτε New\Java Class, ονομάστε την κλάση HelloWorld και πατήστε finish. Προσέξτε πως στο πακέτο σας προστέθηκε ένα αρχείο με το όνομα HelloWorld.java. Παράλληλα, το αρχείο άνοιξε στο τμήμα του editor και είναι έτοιμο για επεξεργασία. Ήδη το NetBeans έχει προσθέσει έναν βασικό κώδικα για να μας βοηθήσει. Πιο συγκεκριμένα, έχει προσθέσει το όνομα του πακέτου στην κορυφή του κώδικα και στη συνέχεια τη δήλωση της κλάσης HelloWorld.

Μετατρέψτε το αρχείο HelloWorld.java όπως δείχνει ο κώδικας 2.1. Αντικαταστήστε όμως το gr.ihu.cs.lmous.OOJ.k1.v1 με το όνομα του δικού σας πακέτου.

```
package gr.ihu.cs.lmous.OOJ.k2.v1;

public class HelloWorld {

    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

***Κώδικας 2.1**: Το πρώτο πρόγραμμα*

Αυτός είναι ο πηγαίος κώδικας του πρώτου μας προγράμματος. Στο τμήμα Projects, κάντε δεξί κλικ πάνω στην κλάση HelloWorld και επιλέξτε Run File. Θα εκτελεστεί το πρόγραμμα με αποτέλεσμα να τυπώσει στο παράθυρο Output τη σειρά χαρακτήρων HelloWorld!. Το NetBeans έκανε αυτόματα μεταγλώττιση του κώδικα σε bytecode και ζήτησε από την JVM να τον τρέξει. Αν μεταβείτε στον folder που έχετε αποθηκεύσει το project, θα διαπιστώσετε πως περιέχει μια σειρά από φακέλους. Μεταβείτε στον φάκελο build\classes\ yourPackagePath (Αν χρησιμοποιήσατε το ίδιο όνομα πακέτου με τον κώδικα 2.1, μεταβείτε στον φάκελο

build\classes\gr\ihu\cs\lmous\OOJ\k1\v1), θα δείτε ότι περιέχει ένα αρχείο HelloWorld.class, πρόκειται για το bytecode της κλάσης μας.

Για να τρέξετε την κλάση σας εκτός παραθυρικού περιβάλλοντος, ανοίξτε ένα κέλυφος του λειτουργικού συστήματος, π.χ. στα Windows ανοίξτε ένα powershell ή ένα command prompt. Μεταβείτε στον φάκελο build\classes, πληκτρολογήστε Java yourPackagePath.HelloWorld και Enter, θα δείτε την έξοδο του προγράμματός σας, δηλαδή στο κέλυφος θα εκτυπωθεί η σειρά χαρακτήρων HelloWorld!

Αυτός είναι σε γενικές γραμμές ο τρόπος με τον οποίο θα δημιουργείτε projects, πακέτα και κλάσεις. Τις κλάσεις βέβαια θα τις τρέχετε κυρίως μέσα από το NetBeans καθώς είναι πολύ απλούστερη διαδικασία. Αργότερα, σε αυτό το βιβλίο, θα δούμε πώς να δημιουργούμε εφαρμογές που συνδυάζουν πολλές κλάσεις, οπότε θα δούμε και έναν διαφορετικό τρόπο για να εκτελούμε τις εφαρμογές.

Ας προχωρήσουμε όμως στην ανάλυση αυτού του βασικού κώδικα. Καταρχάς, κάθε κλάση ανήκει σε ένα πακέτο (package). Το όνομα του πακέτου πρέπει να δηλώνεται απαραιτήτως μαζί με τη δεσμευμένη λέξη package στην πρώτη γραμμή της κλάσης. Λεπτομέρειες για τα πακέτα, τη χρησιμότητά τους και την ονοματολογία τους, μπορείτε να βρείτε στην ενότητα 7.8. Στη συνέχεια ακολουθεί η κλάση. Αυτή ορίζεται με χρήση της λέξης-κλειδί class ακολουθούμενη από το όνομα της κλάσης. Στη συνέχεια ανάμεσα σε μια αγκύλη που ανοίγει και μια που κλείνει, τοποθετούμε τον κώδικα της κλάσης. Προσέξτε πως πριν τη λέξη-κλειδί class, έχει τοποθετηθεί η λέξη-κλειδί public. Η λέξη αυτή καθιστά την κλάση δημόσια, δηλαδή οποιοσδήποτε διαθέτει το αρχείο HelloWorld.class μπορεί να χρησιμοποιήσει την κλάση. Η λέξη public είναι ένας προσδιοριστής προσπέλασης (access specifier). Θα μιλήσουμε αναλυτικότερα για τους προσδιοριστές προσπέλασης στην ενότητα 7.7. Θα πρέπει όμως να πούμε πως σε ένα αρχείο .java μπορεί να τοποθετηθεί μια μόνο public κλάση. Είναι υποχρεωτικό μάλιστα, αρχείο και κλάση να έχουν το ίδιο ακριβώς όνομα.

Μέσα στην κλάση ορίζεται μια συνάρτηση που ονομάζεται main. Για την ώρα, μπορούμε να θεωρήσουμε πως μια συνάρτηση είναι ένα υποπρόγραμμα ή αλλιώς μια ρουτίνα [1] που εκτελείται όταν το ζητήσουμε. Όπως θα δούμε αμέσως παρακάτω σε αυτήν την ενότητα, μπορούμε να ορίσουμε πολλές συναρτήσεις μέσα σε μια κλάση οι οποίες θα εκτελούνται μόνο όταν το ζητήσουμε σαφώς. Η συνάρτηση main διαφέρει. Αποτελεί την κύρια είσοδο στο πρόγραμμά μας. Έτσι όταν ζητάμε από την Java να τρέξει μια κλάση, αυτή ψάχνει να βρει τη συνάρτηση main. Αν η κλάση μας δεν περιλαμβάνει main, τότε δεν είναι εκτελέσιμη. Έτσι είτε τρέξουμε την κλάση μας στο NetBeans είτε την τρέξουμε σε κέλυφος του λειτουργικού, η συνάρτηση εκκίνησης είναι η main. Περισσότερα για την main στην ενότητα 7.3.4. Προσέξτε πως η λέξη main ακολουθείται από τη σειρά χαρακτήρων (String[] args). Πρόκειται για τη λίστα παραμέτρων (parameter list) της main. Δεν θα επεκταθούμε περισσότερο σε αυτό για την ώρα. Εκείνο που πρέπει να γνωρίζουμε είναι ότι η main που αποτελεί είσοδο στο πρόγραμμά μας υποχρεωτικά ακολουθείται από τη συγκεκριμένη λίστα παραμέτρων. Περισσότερα για τις λίστες παραμέτρων των συναρτήσεων γενικά στην ενότητα 7.3 και της main ειδικά στην 7.3.4. Μπροστά από την main έχουμε τοποθετήσει τις λέξεις public static void. Το public έχει επίσης την έννοια πως οποιοσδήποτε διαθέτει το bytecode της κλάσης μπορεί να τρέξει τη συνάρτηση. Το static θα το τοποθετούμε μπροστά από όλες τις συναρτήσεις που θα γράφουμε έως ότου εισαχθούμε στο αντικειμενοστρεφές μοντέλο. Τότε θα δούμε μη στατικές συναρτήσεις, δηλαδή συναρτήσεις που δεν χαρακτηρίζουμε ως static και θα διαφοροποιήσουμε με λεπτομέρεια μεταξύ των δύο. Τέλος, το void είναι ο τύπος της τιμής επιστροφής της συνάρτησης. Περισσότερα για τις τιμές επιστροφής των συναρτήσεων στην ενότητα 7.1.

Ο κώδικας που περιέχεται στην main είναι αυτός που στέλνει στην τυπική έξοδο (standard output), δηλαδή στην οθόνη, την σειρά HelloWorld!. Επομένως, αν θέλουμε να τυπώσουμε μια σειρά χαρακτήρων στην οθόνη, μπορούμε να χρησιμοποιούμε την προκαθορισμένη στην Java συνάρτηση System.out.println ακολουθούμενη από παρενθέσεις μέσα στις οποίες τοποθετούμε την σειρά χαρακτήρων περικλείοντάς την σε διπλά εισαγωγικά. Γενικά, οποτεδήποτε αναφερόμαστε μέσα στο πρόγραμμά μας σε σταθερή σειρά χαρακτήρων θα πρέπει να την εσωκλείουμε σε διπλά εισαγωγικά. Βεβαίως, η System.out.println χρησιμοποιείται για την εκτύπωση και άλλου τύπου δεδομένων και όχι μόνο για σειρές χαρακτήρων.

2.3 Οι συναρτήσεις ως απλά υποπρογράμματα

Παρουσιάζουμε εδώ μια εναλλακτική προσέγγιση για την εκτύπωση της σειράς “HelloWorld”. Θα φτιάξουμε μια νέα κλάση που θα ονομάσουμε ToPrinter. Μέσα στην ToPrinter, θα υλοποιήσουμε μια συνάρτηση που θα ονομάσουμε helloWorld. Στη συνέχεια στην main θα καλέσουμε την helloWorld.

Με δεξί κλικ στο πακέτο σας, δημιουργήστε την κλάση ToPrinter. Διαμορφώστε το περιεχόμενο της ToPrinter όπως φαίνεται στον κώδικα 2.2. Σημειώστε πως στο εξής η ονομασία του πακέτου θα παραλείπεται

από τους κώδικες σε αυτό το βιβλίο. Ωστόσο, για να είναι οι κώδικες μεταγλωττίσιμοι, θα πρέπει να προσθέτετε εσείς τη δήλωση package συνοδευόμενη από το όνομα του δικού σας πακέτου.

```
package gr.ihu.cs.lmous.OOJ.k2.v1;

public class ToPrinter {

    static void helloWorld() {
        System.out.println("HelloWorld!");
    }

    public static void main(String[] args) {
        helloWorld();
    }
}
```

***Κώδικας 2.2**: Η πρώτη έκδοση της κλάσης ToPrinter*

Αν τρέξουμε την κλάση, θα λάβουμε το ίδιο αποτέλεσμα με την εκτέλεση της κλάσης HelloWorld. Όπως βλέπουμε στον κώδικα 2.2, η main καλεί την helloWorld() η οποία με τη σειρά της καλεί την System.out.println που τυπώνει τη σειρά "HelloWorld!". Οι συναρτήσεις είναι υποπρογράμματα ή αλλιώς ρουτίνες και αποτελούν θεμέλιο λίθο τόσο του διαδικαστικού όσο και του αντικειμενοστρεφούς μοντέλου και θα τις χρησιμοποιούμε συστηματικά κατά την ανάπτυξη των κλάσεων. Βεβαίως, η συνάρτηση helloWorld αποτελεί την απλούστερη μορφή συνάρτησης. Είναι μια στατική void συνάρτηση χωρίς παραμέτρους. Εκτεταμένα για τις συναρτήσεις μιλάμε στην ενότητα 7. Για την ώρα, μας αρκεί αυτή η απλή μορφή.

Αν υποθέσουμε τώρα πως θέλουμε να εκτυπώσουμε τη διεύθυνσή μας χρησιμοποιώντας την κλάση ToPrinter, μπορούμε να προσθέσουμε μια κατάλληλη συνάρτηση και να την καλέσουμε στην κλάση όπως δείχνει ο κώδικας 2.3.

```
public class ToPrinter {

    static void helloWorld() {
        System.out.println("HelloWorld!");
    }

    static void address() {
        System.out.println("Ιωάννης Νάκος");
        System.out.println("Εσπερίδων 75");
        System.out.println("Καλλιθέα 65303");
        System.out.println("Αθήνα, Ελλάδα");
    }

    public static void main(String[] args) {
        helloWorld();
        address();
    }
}
```

***Κώδικας 2.3**: Η δεύτερη έκδοση της κλάσης ToPrinter*

Προσέξτε όμως ότι θα πρέπει να τοποθετήσετε τη δεύτερη έκδοση της ToPrinter σε διαφορετικό πακέτο από την πρώτη ή διαφορετικά να αλλάξετε την ονομασία της κλάσης. Τρέχοντας τον κώδικα 2.3, διαπιστώνουμε πως τυπώνεται πρώτα η σειρά "HelloWorld" και στη συνέχεια η διεύθυνση. Ένας εύκολος τρόπος για να τυπωθεί μόνο η διεύθυνση είναι να σχολιάσουμε την κλήση της HelloWorld, δηλαδή να μετατρέψουμε την main όπως δείχνει ο κώδικας 2.4.

```
    public static void main(String[] args) {
        //helloWorld();
        address();
    }
```